PCET'S

Pimpri Chinchwad College of Engineering, Pune

Name: Soham Kulkarni **PRN:** 124B1F056 **Batch:** TY - A3

Course: Design and Analysis of Algorithm Laboratory

ASSIGNMENT NO. 06

1. Problem Statement

After an earthquake, a relief truck must carry essential supplies. Its maximum capacity is W kg. Each package has a weight and a utility value, meaning an importance score. Choose packages that give the highest total utility without exceeding W. A package can be selected once or left behind; it cannot be divided.

2. Course Objectives

• Understand the 0/1 Knapsack problem and apply dynamic programming.

• Compare brute force, dynamic programming and greedy approaches.

• Explain priority, changing supplies and multiple-truck extensions.

3. Course Outcomes

Find an optimal single-truck selection, explain the take-or-leave decision, reconstruct selected packages, and understand the limits of simple extensions.

4. The Basic Idea

0/1 means: 1 = take the package; 0 = leave the package. The truck does not have to be completely full. It must only stay within the weight limit while maximizing utility. Each input row is one indivisible package.

The main program now asks only for the item count, truck capacity, and the name of each item, weight and utility. It has no menus, urgency flags or multiple-truck settings. The original advanced program is preserved separately in the Extensions folder.

5. Understand the Example

There are three items and a truck with capacity **5 kg**. The utility values below are illustrative importance scores, not prices.

Item	Weight	Utility
Medicine	2 kg	30
Food	3 kg	40
Blankets	4 kg	50

Medicine + Food weighs 2 + 3 = **5 kg** and gives utility 30 + 40 = **70**. Medicine + Blankets weighs 6 kg, and Food + Blankets weighs 7 kg, so both are too heavy. Blankets alone gives utility 50. Therefore, Medicine + Food is the best feasible choice.

5.1. How Dynamic Programming Works

dp[i][w] means: the best utility using only the first i items with capacity w. For every item, compare leaving it with taking it once, if it fits.

```
leave = dp[i-1][w]
take = utility[i] + dp[i-1][w - weight[i]]
dp[i][w] = maximum of leave and take
```

For Food at capacity 5: leaving gives 30; taking gives 40 plus the best result for the remaining 2 kg, which is 30. Keep **70**. For Blankets at capacity 5: leaving gives 70; taking gives 50 plus the best result for 1 kg, which is zero. Keep **70** and leave Blankets.

5.2. The DP Table

Items considered	0 kg	1 kg	2 kg	3 kg	4 kg	5 kg
None	0	0	0	0	0	0
Medicine	0	0	30	30	30	30
Medicine, Food	0	0	30	40	40	70
All three	0	0	30	40	50	70

The bottom-right value is the answer, 70. Using the previous row for the take case prevents taking an item more than once. Trace the table backwards: if a value differs from the row above, that item was taken; subtract its weight and continue.

5.3. Algorithm / Pseudocode

```
Read item count N, truck capacity W and item details
Initialize the DP table to zero
For each item i and capacity w:
    Store the answer for leaving item i
    If item i fits, compare with taking it once
Work backwards through the table to recover selected items
Print selected items, total weight and maximum utility
```

6. Sample Input and Output

```
Input:
3 5
Medicine 2 30
Food 3 40
Blankets 4 50

Output:
Selected items:
Name Weight Utility
Medicine 2 30
Food 3 40
Total weight: 5 kg
Unused capacity: 0 kg
Maximum utility: 70
```

7. Approach Comparison

Brute force: try all take-or-leave combinations. Recursive enumeration with running totals takes O(2^N) time and O(N) stack space. It is exact but becomes slow as N grows.

Dynamic programming: fill the (N+1)-by-(W+1) table. Time and space are O(N(W+1)), usually written O(NW) for positive W. This is pseudo-polynomial in numeric capacity. The table also makes selected-item reconstruction easy. The simple program accepts up to 200 items and 10,000 kg, requiring about 16 MB for its largest table.

Ratio greedy: sort by utility/weight, then take fitting items. Sorting takes O(N log N), but the result is not always optimal for 0/1 selection. With capacity 50 and items (10,60), (20,100), (30,120), greedy takes the first two for utility 160. DP takes the last two for utility **220**.

8. Priority and Real-World Extensions

In the simple program, utility is an importance score. Assigning critical supplies a larger utility encourages their selection, but does not force them to be included. The separate advanced program supports explicit urgent-first selection, item availability and updates.

For changing supplies, edit the weights or utilities and rerun the simple program. The advanced extension can also mark expired, missing or dispatched packages unavailable and replan without restarting. Its plans compare alternatives; planning does not actually dispatch inventory.

The advanced program handles multiple trucks sequentially: optimize one truck, remove its selected packages from this plan, and optimize the next. Packages are not duplicated, but the best combined fleet allocation is not guaranteed. An exact fleet extension needs one capacity dimension per truck; its state space grows as the product of the truck capacities plus one.

9. Conclusion

The simple 0/1 Knapsack program compares taking and leaving every item. It gives an exact maximum-utility single-truck selection without overloading the truck. The separate extension retains priority, updates and multiple-truck functionality. C++ code is supplied separately from this report.